

Доклад на тему: Доклад на тему: Проверка неизменности системных файлов и директорий.

Архитектура компьютеров и операционные системы. Раздел:
Операционные системы

Зевакина Екатерина Романовна

16 мая 2026

Российский университет дружбы народов, Москва, Россия

- Зевакина Екатерина Романовна
- Студентка НКАбд-02-25
- Российский университет дружбы народов им. П. Лумумбы



Тема проверки неизменности системных файлов и директорий (File Integrity Monitoring, FIM) — это фундаментальная часть обеспечения безопасности и администрирования UNIX-подобных систем (Linux, FreeBSD, macOS и др.). Простыми словами: это процесс, который позволяет ответить на вопрос — «Не изменился ли какой-либо важный системный файл с момента последней проверки?» Давайте разберем тему системно: зачем, что проверять, как это работает, основные инструменты и что делать с результатами.

Зачем это нужно?

Системные файлы могут измениться по двум причинам: 1. **Легитимная** (разрешенная администратором): - Установка или обновление пакетов (через apt, yum, pkg). - Изменение конфигураций (правка sshd_config, fstab, resolv.conf). - Самостоятельная компиляция и установка софта. 2.

Нелегитимная (опасная): - Взлом (компрометация) — злоумышленник заменяет ls, ps, sshd или libc.so на свои версии (руткиты). - Ошибка администратора (например, chmod 777 /etc/shadow). - Сбой ПО или диска — битые сектора, случайное повреждение. - Внутренний нарушитель.

Цель проверки: обнаружить неавторизованные изменения как можно раньше, особенно на серверах, которые смотрят в интернет.

Что именно проверять?

Нельзя проверять всё — это займет слишком много времени и ресурсов. Обычно в список входят: - **Исполняемые файлы:** /bin, /sbin, /usr/bin, /usr/sbin, /usr/local/bin - **Системные конфигурации:** /etc (почти всё, но особенно passwd, shadow, sudoers, ssh/, pam.d/) - **Библиотеки:** /lib, /lib64, /usr/lib - **Ядро и загрузчик:** /boot, модули ядра (/lib/modules) - **Логи:** чтобы заметить их удаление. - Ваши собственные скрипты или бинарные файлы.

В UNIX хорошая практика — проверять **все, что не является пользовательскими данными** (домашние папки /home обычно исключают, если это не целенаправленная проверка).

Как выполняется проверка? (Хэш-суммы и база знаний)

Идея очень простая, но гениальная: 1. Создание эталонной базы — в чистом, доверенном состоянии системы. - Для каждого проверяемого файла вычисляется криптографическая хэш-сумма. - Эти данные сохраняются в защищенном месте. 2. Регулярная проверка — по расписанию. - Утилита заново вычисляет хэши всех файлов из списка. - Сравнивает их с эталонными. 3. Оповещение — если хэш не совпал, выдается предупреждение: «Файл /bin/ps изменился! Возможно, руткит».

Почему хэш, а не просто дата изменения?

Дату легко подделать (touch), а хэш уникально соответствует содержимому.
Даже 1 байт разницы → хэш полностью меняется.

Классические системные утилиты (ручной способ)

```
#          (          /bin)
find /bin -type f -exec sha256sum {} \; > baseline_bin.txt
#          (          )
sha256sum -c baseline_bin.txt
```

Но у этого способа минусы: - Не хранит метаданные. - Нет логики «ожидаемое изменение». - Медленно для больших директорий.

Специализированные инструменты (рекомендуются)

Инструмент	Особенности	Где работает
AIDE	Самый популярный. Свой конфиг, хранит хэши, права, ссылки. Умеет исключать (/etc/*.~). Есть база данных в текстовом виде.	Linux, BSD, macOS
Tripwire	Классика (1992). Более сложный, но мощный. Разделяет политику и базу. Платный Open Source версия существует.	Linux, Unix
Osquery	Современный инструмент от Facebook. Работает через SQL (SELECT * FROM file WHERE path = '/bin'). Позволяет делать очень гибкие проверки.	Linux, macOS, Windows
Samhain	Может работать как клиент-сервер, шифровать базу, скрывать процесс. Хорош для HIPAA / PCI DSS.	Unix
Integrit	Простой и быстрый, минималистичный.	Unix

Пример с AIDE (классика): 1. Установка: `sudo apt install aide` 2. Настройка (/etc/aide/aide.conf) 3. Инициализация базы: `sudo aideinit` 4. Ежедневная проверка: `aide -check` (есть разница — отправить письмо) 5. После легитимного обновления системы: `sudo aide -update`

Проверка через пакетный менеджер (быстрый способ)

ОС	Команда
Debian / Ubuntu	debsums -c (предварительно установить debsums)
Red Hat / Fedora	rpm -Va (проверит все пакеты)
Arch Linux	paccheck --file-properties (из pacutils)

Пример rpm -Va: - S — размер изменился - M — права/владелец - 5 — MD5-сумма (хэш) - с — конфигурационный файл (обычно ожидаемо) Этот способ хорош для быстрой «поверхностной» проверки.

Что делать при обнаружении изменений?

1. **Выяснить причину**
2. **Сравнить с заведомо эталонной системой**
3. **Проверить файл** на вредоносность:
 - `strings /bin/ps | grep -i "ssh|password|.so"`
 - Проверить через онлайн-сканер VirusTotal.
4. Если это **взлом**:
 - Изолировать сервер от сети (ifconfig down).
 - Не доверять утилитам на этом сервере (руткит может показывать ложные данные).
 - Сравнить с эталоном (который хранится вне сервера).
 - Самый надежный путь: переустановка системы с нуля и восстановление из резервных копий. Пытаться «лечить» взломанный UNIX — часто неблагоприятное занятие.
5. Если это **легитимное изменение** (обновление, правка админом):
 - Зафиксировать новую базу (например, `aide -update`).

1. Храните базу вне проверяемой системы.
2. Запускайте проверку по расписанию
3. Мониторинг неизменности
4. Для конфигураций используйте Git (etckeeper — отличная утилита).
5. Читайте выходной формат.
6. Будьте готовы к ложным срабатываниям.

1. **Что:** сравниваете текущие криптохэши файлов с эталонными.
2. **Зачем:** ловить взломы, ошибки и сбои.
3. **Как:** AIDE, Tripwire, встроенные rpm -Va или debsums.
4. **Главное правило:** эталонная база должна храниться НЕ на проверяемом сервере.

Внедрите эту практику на всех серверах, смотрящих наружу. Это один из столпов защиты глубины (defense in depth) в UNIX-мире.

Список ссылок на сайты, с которых взята информация

Сайт debian

Сайт с командами на Linux

Сайт redhat

Сайт kernel